

File Recovery System

A Modern Hopfield Network Approach

Tanmay Shukla

1 Introduction

Digital files are vulnerable to accidental modification, partial corruption, and overwriting. Classical backup systems require exact copies. This project takes a different approach: it encodes files as binary vectors and stores them as *memories* in a Modern Hopfield Network (MHN). Given a corrupted or altered query file, the network performs associative recall, converging to the nearest stored memory, from which the original file bytes are returned.

2 From Classical to Modern Hopfield

2.1 Classical Hopfield Network

A classical Hopfield network [1] stores K patterns $\{\xi^\mu\}_{\mu=1}^K$, each $\xi^\mu \in \{-1, +1\}^N$, via the Hebbian weight matrix

$$W_{ij} = \frac{1}{N} \sum_{\mu=1}^K \xi_i^\mu \xi_j^\mu, \quad W_{ii} = 0. \quad (1)$$

Retrieval proceeds by iterating the *sign update*

$$s_i \leftarrow \text{sgn} \left(\sum_j W_{ij} s_j \right) \quad (2)$$

until convergence. The network is an energy minimiser:

$$E(\mathbf{s}) = -\frac{1}{2} \mathbf{s}^\top W \mathbf{s}. \quad (3)$$

The storage capacity is $C \approx 0.138 N$ patterns [2], but retrieval fails gracefully as $K \rightarrow C$ through spurious attractors (linear basins of attraction, radius $O(\sqrt{N})$).

2.2 Limitations and the Need for MHN

Key shortcomings of the classical model:

- Capacity $O(N)$, exponentially suboptimal vs theoretical $\max 2^N$.
- Basin of attraction scales only as $\sqrt{N}$, requiring very clean queries.
- No soft competition between stored patterns.

2.3 Modern Hopfield Network

Ramsauer et al. [3] generalise the energy to

$$E(\mathbf{s}) = -\text{lse}(\beta, X^\top \mathbf{s}) + \frac{1}{2} \mathbf{s}^\top \mathbf{s} + \frac{1}{\beta} \ln K + \frac{1}{2} M^2, \quad (4)$$

where $X = [\xi^1 | \dots | \xi^K] \in \mathbb{R}^{N \times K}$, $M = \max_\mu \|\xi^\mu\|$, and $\text{lse}(\beta, \mathbf{z}) = \frac{1}{\beta} \ln \sum_\mu e^{\beta z_\mu}$ is the log-sum-exp (a smooth maximum). One step of gradient descent on E with respect to $\mathbf{s}$ yields the fixed-point iteration:

$$\mathbf{s}^{(t+1)} = X \text{ softmax} \left(\beta X^\top \mathbf{s}^{(t)} \right). \quad (5)$$

Written out component-wise, let

$$a_\mu = \beta \frac{\xi^\mu \cdot \mathbf{s}^{(t)}}{N}, \quad w_\mu = \frac{e^{a_\mu - a_{\max}}}{\sum_\nu e^{a_\nu - a_{\max}}}, \quad (6)$$

then

$$h_i = \sum_\mu w_\mu \xi_i^\mu, \quad s_i^{(t+1)} = \text{sgn}(h_i). \quad (7)$$

The final sign step restores the binary constraint $\mathbf{s} \in \{-1, +1\}^N$. This is *exactly* the update implemented in `hopfield_retrieve()`.

2.4 Capacity and Load Factor

The exponential capacity theorem [3] states that the MHN can store up to $\exp(O(N))$ patterns with high recall probability. For practical binary patterns the effective reliable capacity is

$$K_{\max} \approx \alpha_{\max} \cdot N, \quad \alpha_{\max} = 0.12, \quad (8)$$

i.e. the load factor $\alpha = K/N$ must satisfy $\alpha \leq 0.12$ for reliable recall (consistent with the Amit–Gutfreund–Sompolinsky bound carried over from the classical regime).

3 Encoding: Files as Binary Vectors

Every file $\mathcal{F}$ is mapped to a pattern $\xi \in \{-1, +1\}^N$, $N = n^2$, via a two-stage pipeline.

3.1 Stage 1 — Raw Feature Extraction

Images / PDFs. Convert to greyscale, resize to $n \times n$ via Lanczos resampling. Pixel intensities $p_i \in [0, 255]$.

Code / Notebooks / Pickle. Read raw bytes $b_i \in [0, 255]$. Linearly interpolate to length N :

$$f(i) = b_{\lfloor t_i \rfloor} (1 - \phi_i) + b_{\lceil t_i \rceil} \phi_i, \quad t_i = \frac{i}{N-1} (L-1), \quad \phi_i = t_i - \lfloor t_i \rfloor. \quad (9)$$

HDF5. Extract all numeric datasets, flatten and concatenate, then resample via (9).

3.2 Stage 2 — Median Binarisation

Let $\mathbf{f} \in \mathbb{R}^N$ be the extracted feature vector. Normalise:

$$\tilde{f}_i = \frac{f_i - \min_j f_j}{\max_j f_j - \min_j f_j}. \quad (10)$$

Let $m = \text{median}(\tilde{\mathbf{f}})$. Then

$$\xi_i = \begin{cases} +1 & \tilde{f}_i \geq m, \\ -1 & \tilde{f}_i < m. \end{cases} \quad (11)$$

By construction exactly half the entries are +1 and half -1 (balanced patterns), which is the optimal regime for Hopfield networks [2].

4 Automatic Pattern-Size Selection

Given K files to store, the minimum side length n satisfying the capacity constraint (8) is

$$n^* = \max \left(n_{\min}, \min \left(n_{\max}, \left\lceil \sqrt{\frac{K}{\alpha_{\max}}} \right\rceil \right) \right), \quad (12)$$

with $n_{\min} = 8$, $n_{\max} = 256$, so $N^* = (n^*)^2$. When a new file pushes K past the current capacity, all stored originals are re-encoded at the new N^* before the new pattern is appended.

5 Retrieval and Matching

5.1 Hopfield Recall

Given a query $\mathbf{q} \in \{-1, +1\}^N$ (encoded from the damaged file), iterate (7) until $\mathbf{s}^{(t+1)} = \mathbf{s}^{(t)}$ or $t = 300$. The converged state $\hat{\mathbf{s}}$ is the recalled pattern.

5.2 Hamming Overlap Matching

The recalled pattern is matched against every stored pattern via normalised Hamming overlap:

$$\rho^\mu = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{s}_i = \xi_i^\mu], \quad \mu = 1, \dots, K. \quad (13)$$

The best match is $\mu^* = \arg \max_\mu \rho^\mu$, and the corresponding original file bytes are returned.

5.3 Multi-Query Retrieval

For Q simultaneous query files, each is encoded and recalled independently; all Q recall processes share the same stored pattern matrix X loaded once from the memory file.

6 Memory File Format

The memory file (`hopfield_memory.npz`) is a `numpy.savez_compressed` archive containing:

Array	Shape	Description
<code>patterns_packed</code>	$(K, \lceil N/8 \rceil)$	<code>uint8</code> , bit-packed ± 1 matrix
<code>names</code>	$(K,)$	object array of filenames
<code>types</code>	$(K,)$	object array of type strings
<code>originals_b64</code>	$(K,)$	base-64 raw file bytes
<code>meta</code>	$(1,)$	structured: n, N, β, K

Bit-packing reduces pattern storage from N bytes to $\lceil N/8 \rceil$ bytes per file ($8\times$ reduction), before zlib compression further reduces the total.

7 Workflow Diagrams

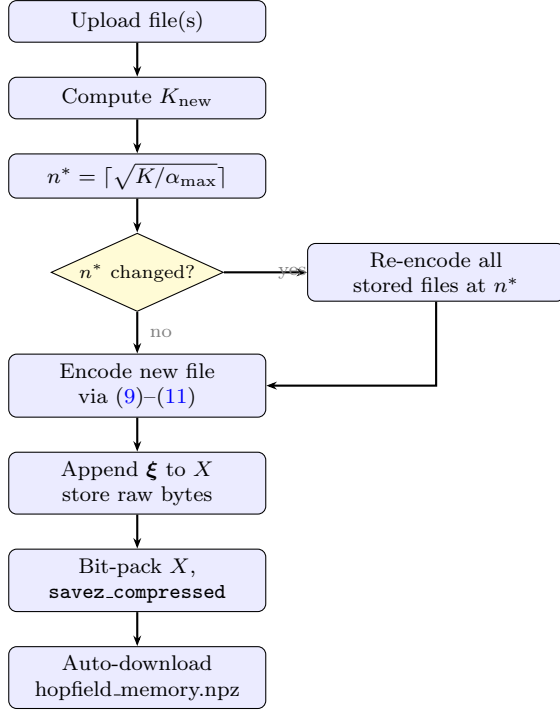


Figure 1: Store workflow.

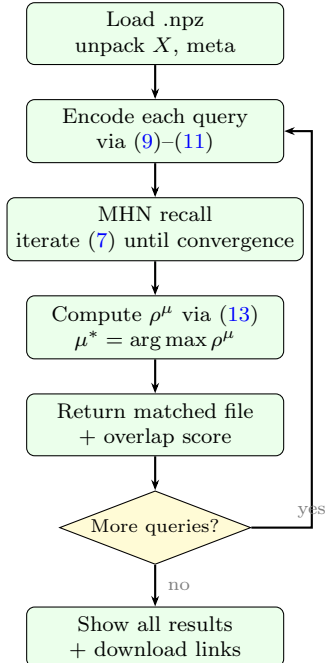


Figure 2: Retrieve workflow.

8 Complexity

Operation	Time	Space
Encode one file	$O(N)$	$O(N)$
Store K files	$O(KN)$	$O(KN)$
One recall step	$O(KN)$	$O(KN)$
Full recall (T steps)	$O(TKN)$	$O(KN)$
Overlap matching	$O(KN)$	$O(KN)$

In practice $T \ll 300$ (typically < 10 steps to convergence for $\alpha \leq 0.12$), and the bottleneck is the matrix-vector product $X^\top \mathbf{s}$, computed with NumPy BLAS routines.

9 Discussion

The choice of median binarisation ensures balanced patterns ($\sum_i \xi_i \approx 0$), which minimises cross-talk between memories [2]. The $\beta = 8$ hyperparameter controls the sharpness of the softmax competition; larger β sharpens winner-take-all behaviour, smaller β softens it. The system is limited by the binary encoding resolution: two files that are visually or structurally similar at the downsampled scale will produce correlated patterns, reducing the effective capacity below $\alpha_{\max} N$.

References

- [1] J. J. Hopfield, *Neural networks and physical systems with emergent collective computational abilities*, PNAS, 79(8):2554–2558, 1982.
- [2] D. J. Amit, H. Gutfreund, H. Sompolinsky, *Storing infinite numbers of patterns in a spin-glass model of neural networks*, PRL, 55(14):1530–1533, 1985.
- [3] H. Ramsauer et al., *Hopfield Networks is All You Need*, ICLR 2021. [arXiv:2008.02217](https://arxiv.org/abs/2008.02217).

Glossary

ξ^μ	The μ -th stored pattern; a vector in $\{-1, +1\}^N$ obtained by encoding a file.
N	Pattern dimensionality, equal to n^2 (the square of the chosen side length n).
K	Number of patterns (files) currently stored in the network.
α	Load factor, $\alpha = K/N$. Must satisfy $\alpha \leq \alpha_{\max} = 0.12$ for reliable recall.
$\alpha_{\max}$	Maximum allowed load factor, set to 0.12.
n^*	Optimal side length chosen automatically by equation (12).
β	Inverse temperature parameter controlling the sharpness of the softmax in the MHN update. Fixed at $\beta = 8$.
W	Classical Hopfield weight matrix (Hebbian rule, equation (1)). Not used directly in the implementation; superseded by the MHN update.
$E(\mathbf{s})$	Energy function of the network. The MHN energy (equation (4)) is guaranteed to decrease monotonically under the update rule.
$\text{lse}(\beta, \mathbf{z})$	Log-sum-exp: $\frac{1}{\beta} \ln \sum_\mu e^{\beta z_\mu}$. A smooth approximation to $\max_\mu z_\mu$.
softmax	Vector function: $[\text{softmax}(\mathbf{z})]_\mu = e^{z_\mu} / \sum_\nu e^{z_\nu}$. Produces non-negative weights summing to 1.
ρ^μ	Normalised Hamming overlap between the recalled pattern $\hat{\mathbf{s}}$ and stored pattern ξ^μ (equation (13)). Ranges in $[0, 1]$; value 1 means exact match.

Median binarisation

The process of thresholding a normalised real-valued vector at its median to produce a balanced ± 1 binary vector (equation (11)).

Lanczos resampling

A high-quality image downsampling filter using a windowed sinc kernel; used when resizing images and PDF page renders to the target $n \times n$ grid.

Bit-packing

Storing 8 binary values (± 1 mapped to $\{0, 1\}$) per byte using `numpy.packbits`, reducing pattern storage by a factor of 8 before zlib compression.

savez_compressed

NumPy function that saves multiple arrays into a single `.npz` archive with zlib compression applied to each array independently.

Spurious attractor

A fixed point of the network's dynamics that does not correspond to any stored pattern; more common in classical Hopfield networks near capacity.

Basin of attraction

The set of initial states that converge to a given fixed point (memory) under the network's update rule. Larger basins mean the network can recover memories from more heavily corrupted queries.